Leonida Ilijoska
Lorenzo Felix
Edoardo Fiorini
Prof. Carlo A. Furia
Programming Fundamentals 1
9th November 2026

Project Proposal for Programming Fundamentals

"Instrument-Based, interactive Flight Simulator"

1. Project Overview

The objective of this project is to develop an interactive, instrument-based flight simulator using Racket.

The simulator will feature an interactive cockpit where users can control key instruments and see how the airplane responds to changes in speed, altitude, and heading.

The simulation will feature a 2D OpenStreetMap navigation map showing the area beneath the plane.

The goal is to create a realistic yet simple flight simulator that is both educational and entertaining.

2. Program Functionality

a. Core Functionalities

The program is based on a mathematical model describing the state of the aircraft. The variables are:

- Position X (m)
- Position Y (m)
- Airspeed (m/s)
- Heading, i.e. direction of travel (degrees)
- Pitch, angle of the nose relative to the horizon (degrees)
- Roll, angle of the wings relative to the horizon (degrees)
- Vertical speed, rate of altitude change (m/s)

- Throttle (0-1), indicator for engine power

The simulator will provide an interactive graphical cockpit displaying the aircraft's current flight conditions (as expressed above).

The cockpit will include an altimeter, airspeed indicator, compass (for heading), artificial horizon (pitch and roll), a vertical speed indicator, and some other basic engine indicators.

Users can control the aircraft through clickable and draggable cockpit controls for pitch, roll, throttle, and heading, with options to pause, resume, or use the keyboard instead.

A mathematical flight model (whose equations are here omitted) will periodically calculate changes in position, airspeed, altitude, heading, pitch, and roll based on the aircraft's controls.

A 2D navigation map will show the aircraft’s position, direction, destination, and surrounding terrain, updating as the plane moves.

Concretely, we will generate a simulated map on which the airplane will be displayed flying as a marker (with the proper orientation).

If time allows, we will integrate OpenStreetMap to dynamically load 2D map tiles as the aircraft moves. We may then use bitmap/url to fetch map data in real time, enabling worldwide flight simulation. The oriented airplane will be placed at the center of the map, which will follow it closely.

The goal is to reach a destination selected by coordinates or by picking a point on the world map, with the simulation confirming successful arrival.

Takeoff and landing will not be required for this basic simulation. In our goals, the whole world map will be covered.

b. Optional Functionalities

If time permits, we will implement a collision detection system that uses altitude data to detect crashes into the ground or mountains, starting with Switzerland and potentially expanding worldwide.

Also, additional features may be implemented:

- **Aerodynamic stalls:** Simulating degraded flight behavior when airspeed falls below a predefined threshold.
- **Terrain proximity warnings:** Alerting the user when approaching dangerous terrain.

- **Audio warnings:** Playing recorded alerts for unsafe flight conditions.
- **Takeoff and landing:** Extending the simulation to support ground operations, acceleration along a runway, takeoff, approach, and landing.

These features will only be considered after the core simulation is functional and tested.

3. Resources and Libraries

a. Programming Language

The project will be developed in Racket using the Intermediate Student with Lambda (ISL+) within the DrRacket development environment.

b. Libraries

The following libraries will be used:

- 2htdp/image: For rendering flight instruments, the navigation map, the aircraft marker, and graphical indicators.
- 2htdp/universe: For handling keyboard inputs, updating the simulation periodically, and managing the interactive interface.
- racket/math: For mathematical calculations, including angle conversions, trigonometric functions, logarithms, and Web Mercator projection.
- sludges: For type annotations and support of the design conventions followed throughout the course.
- map-widget: Provides tools for displaying maps in graphical applications.
- racket/gui/base: playback of audio files for flight warnings “Terrain Terrain” “Stall Stall”

c. Additional Resources

Audio files for the warnings will be supplied externally by us.

4. Main Data Structures

The program will use structures to represent aircraft conditions, geographical obstacles, and the simulation of world state.

Aircraft

Stores the aircraft's position, altitude, airspeed, heading, pitch, roll, vertical speed and throttle.

```
; An Aircraft is a
; (make-aircraft Posn Number Number Number
;                Number Number Number Number)

(define-struct aircraft   (position
altitude airspeed    heading pitch roll
vspeed throttle))
```

FlightStatus

Represents the current simulation condition.

```
; A FlightStatus is one of:
; - "flying"
; - "crashed" #| failed or game over alternatively |#
```

WorldState

Contains the aircraft's current state, its destination, and the simulation status.

```
; A WorldState is a
; (make-world-state Aircraft Posn FlightStatus)

(define-struct world-state
  (aircraft destination status))
```

The WorldState will be updated periodically using big-bang, while the instruments and map will be rendered from its current values.

Instrument

Represents the configuration of a graphical flight instrument, including its type, position on the screen, and display size.

```
; An InstrumentType is one of:
; - "altimeter"
; - "airspeed"
; - "compass" ;
- "horizon"
; - "vertical-speed"
; - "throttle"

; An Instrument is a
; (make-instrument InstrumentType Posn Number)
```

```
(define-struct instrument
  (kind position size))
```

The instrument panel will contain predefined Instrument structures that display readings from the current Aircraft state.

The graphical representation will combine static instrument backgrounds with dynamically rendered elements, such as rotating needles, numerical readings, and moving horizon indicators. Separate rendering functions will handle the specific visual behavior of each instrument.

5. Implementation Scope

The flight simulation will use a simplified mathematical model inspired by Newtonian mechanics. Aircraft motion will be calculated approximating the effects of thrust and drag on airspeed, pitch on vertical movement, and roll and tilting lift on heading.

The implementation will be organized around four principal operations:

```
handle-key(world, key):
    Update aircraft controls
    Return modified world

handle-mouse(world, x, y, event):
    Identify selected cockpit control
    Process button, switch, or slider input
    Return modified world

update-world(world):
    If flying:
        Update airspeed and aircraft motion
        Update altitude, heading, and position
        Check terrain collisions
        Check destination arrival
    Return updated world

render-world(world):
    Draw instruments and interactive controls
    Draw map and aircraft position
    Display flight status
```